Guía de estudio y defensa

RAÍCES CRIOLLAS · Microservicio Pedidos y carrito de compras

Basada en el código Java del ZIP **raices-criollas-backend-dswii-main** entregado en esta conversación. Objetivo: explicar con claridad qué hace cada capa, cómo se conectan los servicios y cómo defender el flujo ante el profesor.

Qué ya probaste: registro/login, carta, carrito (agregar, cambiar cantidades, quitar, vaciar), confirmación del pedido #1 y persistencia en PostgreSQL. El pago efectivo y el descuento posterior de stock no forman parte de estas pruebas.

1. Explicación de 45 segundos para el profesor

“Mi microservicio Pedidos administra el carrito temporal de cada cliente y lo convierte en un pedido persistente. El frontend envía solicitudes REST al Gateway; el controlador de Pedidos obtiene el identificador del usuario autenticado, el servicio aplica validaciones y reglas de negocio, consulta datos de platos en Menú mediante OpenFeign y guarda la información con Spring Data JPA en PostgreSQL. Al confirmar, registra el pedido como PENDIENTE_PAGO, copia la dirección desde Auth y comprueba el stock sin descontarlo. Cuando Payment confirma el pago, Pedidos cambia el estado a PAGADO y emite un evento RabbitMQ para que Menú descuente existencias.”

2. Mapa mental de la arquitectura

```
Navegador / Next.js (localhost:3000)
         | HTTP + JWT
API Gateway (localhost:8080) -- Eureka (descubrimiento)
         |
PedidosController --> PedidosService --> PostgreSQL pedidos_db
                           |--> PlatoClient (Feign) --> Menú
                           |--> AuthClient  (Feign) --> Auth
Payment --> endpoint interno de Pedidos --> RabbitMQ --> Menú (stock)
```

Diferencia importante: Gateway recibe y enruta peticiones; Eureka permite encontrar el destino por nombre; Feign ejecuta llamadas HTTP síncronas; RabbitMQ publica eventos asíncronos. Ninguno sustituye a otro.

3. Estructura que debes ubicar en STS

Paquete/archivo	Responsabilidad
controller/PedidosController.java	Recibe HTTP, solicita identidad, devuelve ApiResponse y códigos HTTP.
service/PedidosService.java	Reglas de carrito, precio, validaciones, creación de pedidos y estados.
entity/Carrito, ItemCarrito	Selección temporal del cliente.
entity/Pedido, DetallePedido	Compra guardada con detalle histórico.
repository/CarritoRepository, PedidoRepository	Acceso JPA a PostgreSQL.
client/PlatoClient, AuthClient	Consultas REST a Menú y perfil de Auth con Feign.
messaging/PedidoEventPublisher	Publica eventos de pedido pagado/cancelado a RabbitMQ.
config/SecurityConfig, RabbitMQConfig	Protección de rutas y configuración de mensajería.
utils/SecurityUtils	Obtiene identidad, rol y Bearer desde contexto JWT.
service/ExpiracionJob	Revisa carritos inactivos y pedidos pendientes vencidos.

4. Ciclo de una solicitud: Controller → Service → Repository

```
@PostMapping("/carrito/items")
public ResponseEntity<ApiResponse<CarritoResponse>> agregar(
        @Valid @RequestBody AgregarItemRequest r) {
  return ResponseEntity.ok(new ApiResponse<>(true, "Plato agregado",
      service.agregar(SecurityUtils.getCurrentUserId(), r)));
}
```

1) **@PostMapping** define la ruta; 2) **@RequestBody** convierte JSON a DTO; 3) **@Valid** valida sus campos; 4) **SecurityUtils** obtiene clienteId del JWT, no del JSON; 5) **service.agregar** ejecuta reglas; 6) **ResponseEntity + ApiResponse** devuelven una respuesta uniforme.

5. Qué ocurre al agregar un producto

• Valida el cliente y que platoId sea positivo.

• Consulta el plato mediante **PlatoClient**: no confía solamente en el nombre o precio enviado por el navegador.

• Busca el carrito por clienteId; si no existe, crea uno.

• Recorre los ítems con un **for**: si encuentra el mismo plato, aumenta cantidad; de lo contrario, crea un ItemCarrito.

• Impide superar **99 unidades por plato** y guarda con **carritos.save(carrito)**.

```
for (ItemCarrito actual : carrito.getItems())
  if (actual.getPlatoId().equals(request.platoId())) {
    item = actual;
    break;
  }
// Si existe, suma cantidad; si no, agrega un nuevo ItemCarrito.
```

6. Métodos REST del carrito

Método / ruta interna	Qué realiza
GET /pedidos/carrito	Consulta carrito del cliente autenticado.
PUT /pedidos/carrito	Reemplaza/sincroniza ítems según la selección enviada.
POST /pedidos/carrito/items	Agrega un plato o incrementa cantidad.
PUT /pedidos/carrito/items/{platoId}	Fija una cantidad exacta.
DELETE /pedidos/carrito/items/{platoId}	Quita un producto.
DELETE /pedidos/carrito	Vacía el carrito.
POST /pedidos/carrito/confirmar	Genera o reutiliza un pedido pendiente.

Estas son rutas del **controlador**; desde el navegador/Bruno se suele acceder a través de las rutas públicas que configura Gateway (p. ej. **/api/pedidos/carrito**).

7. Entidades y relaciones

Carrito → ItemCarrito: un carrito tiene muchos ítems; cada ítem registra un platoId y una cantidad. **Pedido → DetallePedido:** un pedido tiene varios detalles; cada detalle conserva nombre, cantidad, precio unitario y subtotal. **clienteId/platoId** son referencias a otros dominios: no son relaciones JPA entre las bases independientes de Auth, Menú y Pedidos.

¿Por qué conservar precio y nombre en DetallePedido? Porque un cambio futuro en el catálogo no debe alterar el histórico de un pedido. Los cálculos monetarios usan **BigDecimal**, apropiado para valores decimales monetarios.

8. Confirmar: el método más importante

• **confirmar(clienteId)** rechaza un carrito inexistente o vacío.

• Para cada ítem consulta el plato actual, crea DetallePedido, calcula subtotal = precio × cantidad y acumula total.

• **copiarDireccion** consulta el perfil en Auth con el JWT; exige dirección, distrito y teléfono.

• **verificarStock** consulta a Menú; si no hay stock suficiente, se impide continuar.

• Busca pedidos previos PENDIENTE_PAGO: reutiliza uno idéntico y cancela otros pendientes distintos.

• Guarda un pedido con estado **PENDIENTE_PAGO**; no significa que se cobró.

```
BigDecimal total = BigDecimal.ZERO;
for (ItemCarrito item : carrito.getItems()) {
  PlatoClient.PlatoData plato = consultarPlato(item.getPlatoId());
  DetallePedido detalle = new DetallePedido();
  detalle.setPrecioUnitario(plato.precio());
  detalle.setCantidad(item.getCantidad());
  detalle.setSubtotal(plato.precio().multiply(
      BigDecimal.valueOf(item.getCantidad())));
  total = total.add(detalle.getSubtotal());
  pedido.getDetalles().add(detalle);
}
```

Matiz muy importante del código actual: confirmar() NO vacía el carrito directamente. El carrito se elimina en **marcarPagado()** al confirmarse el pago, o cuando el cliente utiliza explícitamente la operación **vaciar()**. No confundas esta versión con la implementación anterior del proyecto.

9. Feign: comunicación con Menú y Auth

```
@FeignClient(name = "raices-criollas-menu")
public interface PlatoClient {
  @GetMapping("/platos/{id}")
  MenuResponse obtenerPlato(@PathVariable("id") Long id);
  @PutMapping("/platos/stock/verificar")
  void verificarStock(@RequestHeader("Authorization") String bearer,
                      @RequestBody List<ItemStock> items);
}
```

PlatoClient obtiene nombre, precio, disponibilidad e imagen, y verifica stock. **AuthClient** consulta **/api/auth/me** para dirección y teléfono. Los nombres de @FeignClient se resuelven mediante Eureka. Una llamada Feign espera respuesta HTTP (síncrona).

10. Seguridad y JWT

Auth autentica al usuario y entrega un JWT. El frontend adjunta **Authorization: Bearer ...** en rutas protegidas. Spring Security valida el token; **SecurityUtils.getCurrentUserId()** obtiene el cliente del contexto de seguridad. Así un cliente no elige arbitrariamente otro clienteId en el cuerpo de la petición. Las rutas de administración y la comunicación interna tienen protecciones configuradas en SecurityConfig; consulta allí los permisos exactos si el profesor lo pregunta.

11. ¿Qué sucede cuando se paga?

Payment comunica el pago aprobado mediante **PUT /pedidos/interno/{id}/pagado**. **marcarPagado()** bloquea el pedido para actualizarlo, evita repetir operaciones si ya no está PENDIENTE_PAGO, cambia a PAGADO, elimina el carrito del cliente y publica un evento con **PedidoEventPublisher**. Menú consume el evento RabbitMQ para ajustar stock. Es una arquitectura de eventos: el descuento no sucede al añadir al carrito ni simplemente al confirmar.

Estados definidos: PENDIENTE_PAGO → PAGADO → PREPARANDO → LISTO → ENVIANDO → ENTREGADO. También existe CANCELADO; **EstadoPedido.puedePasarA()** controla transiciones. El administrador no puede marcar PAGADO manualmente mediante cambiarEstado: lo informa el sistema de pagos.

12. Anotaciones y conceptos que suelen preguntar

Concepto	Explicación sencilla
@Transactional	Agrupa cambios de base de datos en una transacción local; si hay error, se revierte el trabajo DB correspondiente.
@EntityGraph(attributePaths="items")	Al consultar un Carrito, trae sus items asociados, evitando depender de una carga diferida posterior.
JpaRepository	Ya incluye save, findById y delete; Spring Data deriva consultas del nombre del método.
Optional<Carrito>	Expresa que podría no haber carrito para ese cliente.
record DTO	Objeto de datos conciso e inmutable superficialmente para entrada/salida.
@Valid	Solicita ejecutar Bean Validation sobre el DTO.
@PathVariable	Extrae un ID que llega por la URL.
@RequestBody	Convierte cuerpo JSON a objeto Java.
@Scheduled	Ejecuta revisiones periódicas de expiración de pedidos/carritos.

13. Verificación real realizada en PostgreSQL

En la prueba observada se consultó **pedidos_db** en el contenedor **raices-criollas-postgres-pedidos**. El pedido 1 quedó registrado con clienteId 5, estado PENDIENTE_PAGO y total S/ 62.40. Sus cuatro detalles son Alitas BBQ (18.50), Arroz Chaufa (22.50), Ensalada Mediterránea (16.90) y Coca Cola 500ml (4.50); cantidades 1. La suma es 62.40. No se comprobó el pago final.

```
docker exec -it raices-criollas-postgres-pedidos \
  psql -U postgres -d pedidos_db
SELECT id, cliente_id, estado, total FROM pedidos WHERE id = 1;
SELECT nombre_plato, cantidad, precio_unitario, subtotal
FROM detalles_pedido WHERE pedido_id = 1;
SELECT * FROM carritos WHERE cliente_id = 5;
\q
```

14. Preguntas que puede hacerte el profesor

Pregunta: ¿Cuál es tu aporte?
Respuesta: Implementación y verificación del dominio Pedidos: carrito, confirmación, detalle, estados y persistencia.

Pregunta: ¿Por qué separar carrito y pedido?
Respuesta: El carrito es editable y temporal; el pedido es el registro histórico que continúa al pago.

Pregunta: ¿Qué evita que el cliente manipule el precio?
Respuesta: El precio se vuelve a consultar en Menú durante la confirmación, no se toma directamente del navegador.

Pregunta: ¿Para qué sirve Feign?
Respuesta: Permite invocar endpoints de Menú y Auth desde interfaces Java, resolviendo servicios registrados en Eureka.

Pregunta: ¿Por qué RabbitMQ?
Respuesta: Permite avisar de cambios de stock mediante eventos desacoplados después de confirmar el pago.

Pregunta: ¿Se descuenta stock al agregar?
Respuesta: No. La comprobación de stock es previa; el descuento se emite al pagarse.

Pregunta: ¿Por qué BigDecimal?
Respuesta: Para operaciones monetarias decimales precisas.

Pregunta: ¿Qué hace @Transactional?
Respuesta: Hace atómica la operación local de persistencia y permite rollback ante fallo.

Pregunta: ¿Cómo se obtiene clienteId?
Respuesta: Desde el JWT validado, mediante SecurityUtils, en vez de fiarse del JSON del usuario.

Pregunta: ¿Por qué aparece PENDIENTE_PAGO?
Respuesta: Porque el pedido ya está registrado, pero aún espera confirmación de Payment.

Pregunta: ¿Qué ocurre si repito la confirmación?
Respuesta: Si existe un pedido pendiente idéntico, el servicio puede reutilizarlo y renovar su fecha.

Pregunta: ¿Cuándo se vacía el carrito?
Respuesta: En el código actual al marcar el pedido pagado o al invocar vaciar, no automáticamente en confirmar.

Pregunta: ¿Cómo probaste persistencia?
Respuesta: Consulté pedidos y detalles_pedido con psql dentro del contenedor PostgreSQL y verifiqué id, estado y total.

Pregunta: ¿Qué hace un @EntityGraph?
Respuesta: Carga junto al carrito la colección de ítems relacionada para la consulta.

15. Mini-guion de demostración (5 minutos)

• Abrir Eureka y señalar Pedidos registrado; explicar el recorrido frontend → Gateway → servicio.

• Iniciar sesión como cliente; explicar que el JWT identifica al usuario.

• Abrir carta, agregar 2 productos, cambiar cantidad y quitar uno; mostrar carrito y total.

• Confirmar un pedido de prueba con dirección registrada y mostrar su estado PENDIENTE_PAGO.

• Consultar tablas pedidos y detalles_pedido con psql; demostrar persistencia y detalle histórico.

• Señalar que Mercado Pago es otro servicio; explicar marcarPagado y evento RabbitMQ sin ejecutar cobros reales.

16. Archivo exacto que debes estudiar primero

Orden recomendado: **PedidosController.java** → **PedidosService.java** (agregar, reemplazarCarrito, actualizarCantidad, eliminarItem, vaciar, confirmar, marcarPagado) → **Carrito/ItemCarrito/Pedido/DetallePedido** → **Repositorios** → **PlatoClient/AuthClient** → **SecurityConfig/SecurityUtils** → **PedidoEventPublisher/EstadoPedido**. Abre los archivos del ZIP vigente, no una rama antigua.

Fuente y alcance: código fuente aportado, en raices-criollas-pedidos/src/main/java/com/raicescriollas/pedidos/; evidencias de navegador y psql compartidas en esta conversación. Esta guía explica el código y las pruebas observadas; no asegura que el pago real, webhooks o todos los casos límite hayan sido probados.